Medical Image Classification Using Convolutional Neural Network with Keras

Syahla Maharani Salsabila
Department of Biomedical Engineering
Telkom University
Bandung, Indonesia
maharanisyahla@gmail.com

***Abstract*—This paper explores medical image classification using Convolutional Neural Networks (CNNs) with the Keras framework. It presents an effective CNN architecture specifically designed for medical image classification, elaborates on the training and evaluation process of the CNN model, and showcases the outcomes of the experiments conducted on an available dataset. The experimental results demonstrate that the proposed CNN model achieves remarkable accuracy in classifying medical images.**

***Index Terms*—CNN, Image Classification, Keras, Medical Image**

I. Introduction

Accurate interpretation of medical images from modalities such as CT-Scan or MRI, is crucial for health professionals to make exact diagnosis. One of the methods for interpreting medical images is through classification. The purpose of image classification is to classify input images into certain categories [1]. Manual medical image classification is highly time-consuming, requires a lot of expenses, and limiting healthcare professionals' ability to diagnose a high volume of patients. Therefore, effective medical images classification plays an important role in supporting patient care and treatment. For example, analyzing X-ray images through classification is the best approach to diagnose pneumonia which causes about 50,000 people to die per year in the US, but classifying pneumonia from chest X-rays needs professional radiologists which is a rare and expensive resource for some regions [2]. Thus, a solution is needed to make medical image classification more effective and precise.

Convolutional Neural Network (CNN) is being widely accepted and practiced as effective tools for image understanding due to their ability to learn and extract features automatically [3]. CNN also has been proved successful in solving image classification problems [4]. CNN is a deep learning algorithm that can train large datasets with millions of parameters and take 2D image shapes as input, then combine them with filters to produce the desired output. CNNs aim to learn the hierarchical structure of spatial elements automatically and adaptively by using backward propagation of multiple building blocks (such as convolutional layers, composite layers, and fully connected layers) [5]. Therefore, CNN can be an appropriate model option for medical image classification. An example of a CNN structure is shown in Fig. 1 [6]

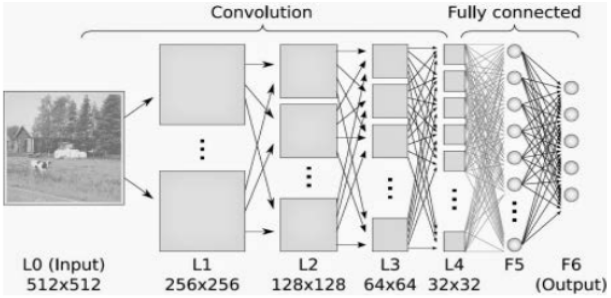

Fig. 1. Convolutional Neural Network.

Deep learning frameworks empower developers to iterate and test prototype models quickly, thus accelerating the machine learning development process. Keras stands as a widely adopted deep learning framework, valued for providing a high-level interface for building and training deep models learning [7]. Thus, Keras can be used to optimize the designed medical image classification model.

This paper will discuss medical image classification using CNN with Keras. First, a brief review of relevant prior research is given. Subsequently, the methodology to create a CNN architecture for medical image classification will be elaborated. Following this, the training and evaluation process of the CNN model will be explained. Finally, the training results obtained from the available dataset will be presented.

II. Related Works

Traditional machine learning techniques, such as Support Vector Machines (SVMs), have long been used in medical images classification. These techniques do, however, have disadvantages: their performance often falls short of practical requirements, the feature extracting and selection are time-consuming and vary according to different objects, and advancements in these methods have stagnated in recent years. In the other hand, CNN is widely used in changing image classification tasks and have achieved significant performance since 2012 [2]. Many research on medical image classification by CNN has achieved excellent performances.

Sonavane, et al. [8] implemented a CNN model for dental cavity classification in their research. They used a Kaggle teeth dataset containing only 74 images, with a 60/14 train-test

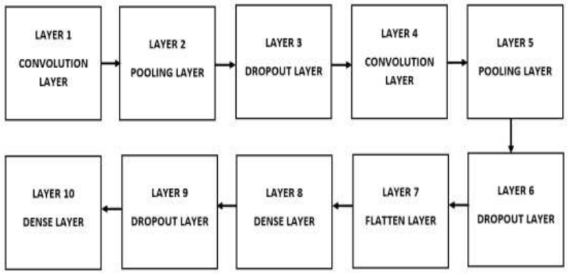

Fig. 2. Layers in Sonavane et al.'s CNN model.

split. CNN model in their research was created with sequential model which contains 10 layers (as shown in Fig. 2), binary cross entropy loss with learning rate of 0.001, and 30 epochs. The model was tested by tuning hyperparameters and achieved a maximum accuracy of 71.43%. Furthermore, Sonavane, et al. stated that accuracy can be increased by increasing the dataset images. Sonavane et al.'s work demonstrates the potential of CNNs for medical image classification, particularly in the field of dental cavity, despite the limited dataset.

III. Method

To achieve the assignment objectives, this study employed a process segmented into the following key stages:

A. Dataset

For this assignment, we utilized the periodontal_disease dataset, a collection of human gum images categorized as either inflammation or normal. This dataset was processed using a CNN-Keras AI model to establish the characteristics of healthy and inflamed human's gum.

Fig. 3. Snapshot of the dataset used.

The periodontal_disease dataset is strategically divided into training, testing, and validation sets to optimize the development of the CNN-Keras model. The training set, comprising 132 images (82 labeled as inflammation and 50 labeled as normal), provides the foundation for the model's learning process. The test set, containing 44 images (28 labeled as inflammation and 16 labeled as normal), serves to assess the model's generalizability on unseen data. Finally, the validation set, also with 44 images (mirroring the class distribution of the test set), is used for hyperparameter tuning and preventing overfitting during training.

```
[ ] train_data_dir = '/content/drive/MyDrive/periodontal_disease/periodontal_disease/train'
    validation_data_dir = '/content/drive/MyDrive/periodontal_disease/periodontal_disease/val'
    test_data_dir = '/content/drive/MyDrive/periodontal_disease/periodontal_disease/test'
    nb_train_samples = 132
    nb_validation_samples = 44
    epochs = 20
    batch_size = 4

    img_width, img_height = 256, 256
```

Fig. 4. Dataset Directory Paths in Google Colab.

The provided function (as shown in Fig. 5) served as a crucial component for image classification tasks. It facilitated the creation of a data generator specifically designed to load and pre-process images from various directories.

```
def make_data_generator(
    train_datagen, train_data_dir,
    val_datagen, validation_data_dir,
    test_datagen, test_data_dir, # Anda perlu mendefinisikan test_data_dir
    batch_size):

    datagen_lst = [train_datagen, val_datagen, test_datagen]
    directory_lst = [train_data_dir, validation_data_dir, test_data_dir]
    generator_lst = []
    for generator, directory in zip(datagen_lst, directory_lst):
        if directory == train_data_dir:
            shuffle = True
        else:
            shuffle = False
        g = generator.flow_from_directory(directory = directory,
                                          target_size = (img_width, img_height),
                                          batch_size = batch_size,
                                          class_mode = 'binary',
                                          shuffle = shuffle,
                                          seed = 42
                                          )
        generator_lst.append(g)

    return generator_lst
```

Fig. 5. Code snippet for creating data generators.

B. Image Pre-processing

To optimize the training process, the images went through pre-processing prior to model training. This enhanced training efficiency by reducing computational workload, accelerating training time, and ultimately improving model accuracy [9]. In this assignment, the following image pre-processing steps were performed.

```
if K.image_data_format() == 'channels_first':
  input_shape = (3, img_width, img_height)
else:
  input_shape = (img_width, img_height, 3)
```

Fig. 6. Code snippet for checking the image data format.

a) Checking The Image Data Format: :Pre-processing is crucial as CNN models require their input data to match a specific format. Defining the input shape ensures the model correctly processes images with the appropriate number of color channels and spatial dimensions (height and width). The

code snippet to check the image data format is shown in Fig. 6.

```
train_datagen = ImageDataGenerator(
    rescale=1.0/255.0,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True)

val_datagen = ImageDataGenerator(rescale=1.0/255.0)
test_datagen = ImageDataGenerator(rescale=1.0/255.0)

train_generator, validation_generator, test_generator = make_data_generator(
    train_datagen, train_data_dir,
    val_datagen, validation_data_dir,
    test_datagen, test_data_dir,
    batch_size)
```

Fig. 7. Code snippet for data augmentation.

b) Data Augmentation: : To enrich the training dataset and enhance model, data augmentation is implemented using Keras' ImageDataGenerator class from the 'preprocessing.image' module. The data augmentation process utilizes various techniques to expand the training set. These techniques include image rescaling, random shearing, zooming, and horizontal flipping. Finally, all augmented images are resized to ensure they match the pre-defined input shape required by the CNN model. The code snippet for data augmentation is shown in Fig. 7.

C. CNN Model

To achieve excellent image classification, a CNN architecture is constructed using Keras' Sequential API. The model combines a sequence of convolutional, activation, pooling, dropout, flatten, and dense layers, as will be described in detail. This model was created with 10 layers sequential. The model was compiled with Adam optimizer with learning rate of 1e-4, binary cross entropy loss function, and accuracy metrics. Code snippet for CNN model is shown in Fig. 8.

```
# CNN model
model = keras.Sequential([
            keras.layers.Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(256, 256, 3)),
            keras.layers.MaxPooling2D((2, 2)),
            keras.layers.Dropout(0.25),

            keras.layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
            keras.layers.MaxPooling2D((2, 2)),

            keras.layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
            keras.layers.MaxPooling2D((2, 2)),

            keras.layers.Flatten(),

            keras.layers.Dense(128, activation='relu'),
            keras.layers.Dense(1, activation='sigmoid')
        ])

# Compile the model
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate = 1e-4),
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.summary()
```

Fig. 8. Code snippet for CNN model.

- Layer 1: Conv2D layer with 32 filters, kernel size (3, 3), 'same' padding , and 'relu' activation. This is the first layer that received input of the form (256, 256, 3).
- Layer 2: MaxPooling2D layer with pooling size (2, 2). It reduced the spatial dimension of the input by maximizing the values in the moving window.
- Layer 3: Dropout layer with a loss rate of 0.25. It shuffled dropout or turned off some input units during training.
- Layer 4: Conv2D layer with 64 filters, kernel size (3, 3), 'same' padding, and 'relu' activation.
- Layer 5: MaxPooling2D layer with pooling size (2, 2).
- Layer 6: Conv2D layer with 128 filters, kernel size (3, 3), 'same' padding, and 'relu' activation.
- Layer 7: MaxPooling2D layer with pooling size (2, 2).
- Layer 8: Flatten layer whose function was to convert the input shape into 1D.
- Layer 9: Dense layer with 128 units and 'relu' activation.
- Layer 10: Dense layer with 1 unit and 'sigmoid' activation. This layer generated the classification probability.

Model summary of CNN architecture is shown in Fig. 9.

```
Model: "sequential_4"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d_12 (Conv2D)          (None, 256, 256, 32)      896

 max_pooling2d_12 (MaxPooli  (None, 128, 128, 32)      0
 ng2D)

 dropout_4 (Dropout)         (None, 128, 128, 32)      0

 conv2d_13 (Conv2D)          (None, 128, 128, 64)      18496

 max_pooling2d_13 (MaxPooli  (None, 64, 64, 64)        0
 ng2D)

 conv2d_14 (Conv2D)          (None, 64, 64, 128)       73856

 max_pooling2d_14 (MaxPooli  (None, 32, 32, 128)       0
 ng2D)

 flatten_4 (Flatten)         (None, 131072)            0

 dense_8 (Dense)             (None, 128)               16777344

 dense_9 (Dense)             (None, 1)                 129

=================================================================
Total params: 16870721 (64.36 MB)
Trainable params: 16870721 (64.36 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

Fig. 9. Model summary of CNN architecture.

D. Train The CNN Model

With the CNN architecture for image classification established, the next step was to train the model. This crucial stage empowered the model to acquire the ability to identify and learn the key characteristics of medical images. Through this training, the model developed the capability to map these learned features to their appropriate classification labels. The training method used is fit_generator() with batch sizes of 4 and 20 epoches. The fit_generator() method trained the model using backpropagation and stochastic gradient descent. This method updated the weights and biases of the model after each set of training data is processed.

```
history = model.fit_generator(
    train_generator,
    steps_per_epoch=nb_train_samples // batch_size,
    epochs=epochs,
    validation_data=validation_generator,
    validation_steps=nb_validation_samples // batch_size)

Epoch 1/20
<ipython-input-85-50dac1443f25>:1: UserWarning: `Model.fit_generator` is deprecated and will be removed in a future version. Please u
  history = model.fit_generator(
33/33 [==============================] - 66s 2s/step - loss: 0.7230 - accuracy: 0.5985 - val_loss: 0.6835 - val_accuracy: 0.6364
Epoch 2/20
33/33 [==============================] - 39s 1s/step - loss: 0.6685 - accuracy: 0.6212 - val_loss: 0.6609 - val_accuracy: 0.6364
Epoch 3/20
33/33 [==============================] - 41s 1s/step - loss: 0.7056 - accuracy: 0.6212 - val_loss: 0.6678 - val_accuracy: 0.6364
Epoch 4/20
33/33 [==============================] - 38s 1s/step - loss: 0.6842 - accuracy: 0.5379 - val_loss: 0.6777 - val_accuracy: 0.6364
Epoch 5/20
33/33 [==============================] - 40s 1s/step - loss: 0.6603 - accuracy: 0.6364 - val_loss: 0.6680 - val_accuracy: 0.6364
```

Fig. 10. Code snippet for training the model.

IV. Results

This section assesses the effectiveness of the trained CNN model in accurately classifying medical images. A comprehensive set of metrics, including accuracy, precision, recall,

specificity, and F1-score, is employed to evaluate the model's performance. During the training process, the experiment used batch sizes of 4 and 20 epoches. In this assignment, the dataset had a total of two classes, "Normal" and "Inflammation".

Based on [10], there are four terms used to describe the performance of a classifier in a binary classification task.

- True Positive (TP): True Positive signifies an instance where the model accurately predicts the presence of the positive class. In this specific case, True Positive refers to an image exhibiting inflammation that is correctly classified as "Inflammation."
- True Negative (TN): True negatives signifies an instance where the model correctly predicts the negative class. In this case, True negative indicates "Normal" which was correctly identified as "Normal".
- False Positive (FP): False Positive signifies an error where the model incorrectly predicts the presence of the positive class in an image that belongs to the negative class. In this case, False positive indicates "Normal" which was misclassified as "Inflammation".
- False Negative (FN): False Positive signifies an error where the model incorrectly predicts the presence of the negative class in an image that belongs to the positive class. In this case, False negative indicates "Inflammation" which was misclassified as "Normal".

A. Confusion Matrix

Confusion matrix (as shown in Fig. 11) illustrates the comparison of the classification results performed by the model with the actual classification results.

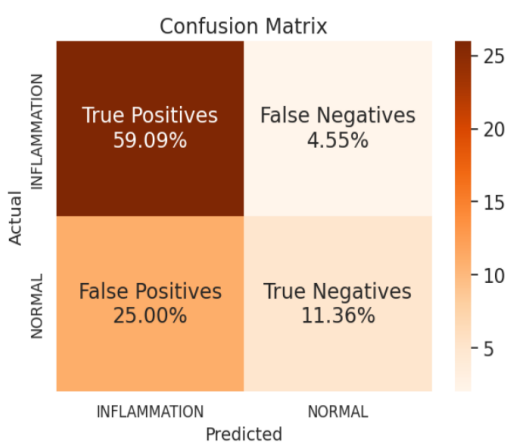

Fig. 11. Confusion Matrix from CNN-Keras Model.

B. Accuracy

Accuracy is the percentage of images that are correctly classified by the model [9]. It was calculated by dividing the number of correct predictions by the total number of predictions, as shown in (1).

$$Accuracy = \frac{TP + TN}{TotalPrediction} \tag{1}$$

In this assignment, the accuracy was achieved at 70.45%. The accuracy plot from this model is shown in Fig. 12.

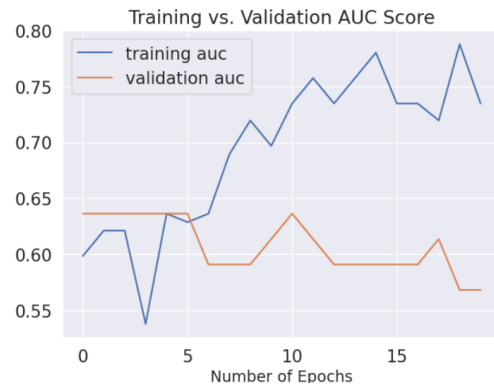

Fig. 12. Accuracy plot from the CNN-Keras Model.

C. Train and Validation Loss

The train and validation loss plot from this model is shown in Fig. 13.

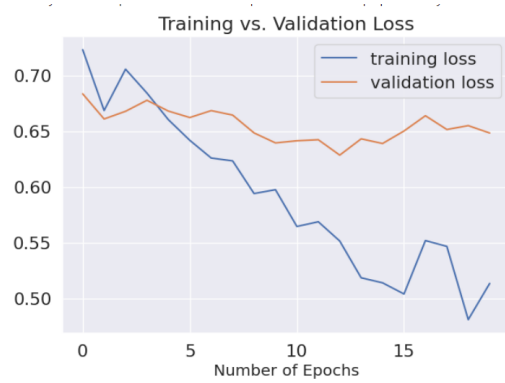

Fig. 13. Train and validation loss plot from the CNN-Keras Model.

D. ROC Curve

The Receiver Operating Characteristic (ROC) curve represents the relation between sensitivity (true positive rate) and specificity (false positive rate) of a classification model. The ROC Curve plot from this model is shown in Fig. 14.

E. Precision

Precision is the percentage of predicted positive cases that are positive [9]. It was calculated by dividing the number of true positive predictions by the total number of positive predictions, as shown in (2). In this assignment, the precision was achieved at 70.20%.

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

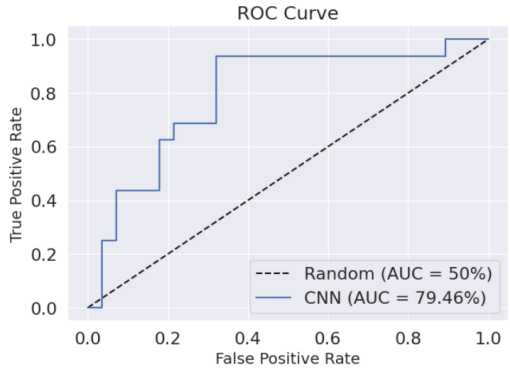

Fig. 14. ROC curve plot from the CNN-Keras Model.

F. Recall

Recall is the percentage of actual positive cases that are correctly predicted by the model [9]. It was calculated by dividing the number of true positive predictions by the total number of actual positive cases, as shown in (3). In this assignment, the recall was achieved at 92.86%.

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

G. F1 Score

F1 Score is the harmonic mean of precision and recall [9]. It was calculated by taking the average of the precision and recall, with higher weights given to lower values, as shown in (4). In this assignment, the F1 Score was achieved at 80.00%.

$$F1Score = 2 * \frac{Precision * Recall}{Precision + Recall} \tag{4}$$

H. Prediction Results

The code snippet for predicting images is shown in Fig. 15 and Fig. 16.

```
# Load model yang telah disimpan
model = load_model('/model/cnn.h5')

# Fungsi untuk memprediksi gambar
def predict_image(img_path):
    img = tf.keras.preprocessing.image.load_img(img_path, target_size=(256, 256))
    img_tensor = tf.keras.preprocessing.image.img_to_array(img)  # Image data encoded as integers in the 0-255 range
    img_tensor /= 255.  # Normalize to [0,1] for plt.imshow application

    # Mengubah bentuk menjadi (1, 256, 256, 3)
    img_tensor = np.expand_dims(img_tensor, axis=0)

    # Prediksi gambar
    prediction = model.predict(img_tensor)
    return prediction
```

Fig. 15. Code snippet for predicting images (1).

The example of the prediction results from this model is shown in Fig. 17, with probabilities in the range of 50-60%.

V. Conclusion

In the domain of computer vision, image classification refers to the automated assignment of labels to images from a pre-defined set of categories. CNNs represent a powerful approach for handling this task, and the Keras framework provides a user-friendly platform for building and deploying

```
# List of test paths
test_paths = ['/content/drive/MyDrive/periodontal_disease/periodontal_disease/test/inflammation/n00000003_000197.jpg',
              '/content/drive/MyDrive/periodontal_disease/periodontal_disease/test/inflammation/n00000003_000163.jpg',
              '/content/drive/MyDrive/periodontal_disease/periodontal_disease/test/normal/n00000003_000190.jpg',
              '/content/drive/MyDrive/periodontal_disease/periodontal_disease/test/normal/n00000003_000198.jpg']

# Loop over each test path
for test_path in test_paths:
    # Prediksi gambar
    prediction = predict_image(test_path)

    # Tampilkan gambar
    img = tf.keras.preprocessing.image.load_img(test_path, target_size=(256, 256))
    plt.imshow(img)
    plt.show()

    # Tampilkan hasil prediksi
    if prediction < 0.5:
        print("Gambar ini diprediksi sebagai 'INFLAMMATION' dengan probabilitas {:.2%}".format(1 - prediction[0][0]))
    else:
        print("Gambar ini diprediksi sebagai 'NORMAL' dengan probabilitas {:.2%}".format(prediction[0][0]))
```

Fig. 16. Code snippet for predicting images (2).

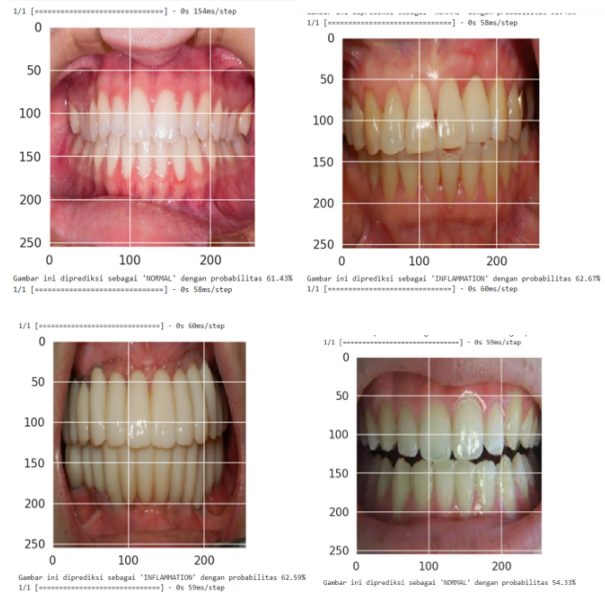

Fig. 17. The example of the prediction results.

CNN models. Notably, CNN-Keras models have demonstrated the ability to achieve quite high accuracy, precision, and recall in various image classification applications. This success highlights the effectiveness and reliability of CNN-Keras for image classification tasks.

References

[1] M. Nurhadi and J. Purnomo, "IMPLEMENTATION OF IMAGE CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORK (CNN) ALGORITHM ON VEHICLES IMAGES," ASEAN Journal of System Engineering, vol. 6, no. 1, pp. 1-5, 2022.

[2] S. S. Yadav and S. M. Jadhav, "Deep convolutional neural network based medical image classification for disease diagnosis," Journal of Big Data, vol. 6, no. 113, 2019.

[3] F. A. Mohammed, K. K. Tune, B. G. Assefa, M. Jett and S. Muhie, "Medical Image Classifications Using Convolutional Neural Networks: A Survey of Current Methods and Statistical Modeling of the Literature," Machine Learning and Knowledge Extraction, vol. 6, no. 1, pp. 699-735, 2024.

[4] Q. Li, W. Cai, X. Wang, Y. Zhou, D. D. Feng and M. Chen, "Medical Image Classification with Convolutional Neural Network," in 13th International Conference on Control Automation Robotics and Vision (ICARCV), Singapore, 2014.

[5] A. S. Adi, "Konsep Dasar Convolutional Neural Network (CNN)," Warung Sains Teknologi, 2024 February 2024. [Online]. Available: https://warstek.com/convolutional-neural-network/. [Accessed 25 April 2024].

[6] M. Tummala, ”Image Classification Using Convolutional Neural,” International Journal of Scientific and Research Publications, vol. 9, no. 8, pp. 382-385, 2019.
[7] T. Suprapto, et al, Modul Praktikum Pemrograman Berbasis Objek dan Aplikasi, Bandung: Telkom University, 2024.
[8] A. Sonavane, R. Yadav and A. Khamparia, ”Dental cavity Classification of using Convolutional Neural Network,” in 1st International Conference on Computational Research and Data Analytic, Rajpura, India, 2020.
[9] L. A. Susanti, A. M. Soleh and B. Sartono, ”DEEP LEARNING IMAGE CLASSIFICATION RONTGEN DADA PADA KASUS COVID-19 MENGGUNAKAN ALGORITMA CONVOLUTIONAL NEURAL NETWORK,” Jurnal Teknologi Informasi dan Ilmu Komputer, vol. 10, no. 5, pp. 973-982, 2019.
[10] S. Verma, K. Anand, K. Verma, D. Gupta and K. Saxena, ”APPLICATION OF CNN MODEL ON MEDICAL IMAGE,” International Research Journal of Engineering and Technology, vol. 10, no. 5, pp. 1630-1634, 2023.